Сценарий видеозащиты PasswordGenerator 1.0.0

Ориентир - 5:30. Требование - не менее 5 минут. До записи заполните ФИО и фактическую дату. Подсказки к демонстрации не зачитываются.

0:00–0:35. Вступление
Здравствуйте, меня зовут _____, дата проведения итоговой аттестации –_____, программа обучения –«Python-разработчик с использованием инструментов ИИ», период обучения с 05.10.2026 по 09.11.2026.
Тема моей итоговой работы: «Создание генератора случайных паролей с оценкой сложности».
Я представляю приложение PasswordGenerator версии 1.0.0. Оно работает локально и имеет графический интерфейс на Tkinter.

На экране: Показать титульный слайд и своё лицо. Заполнить ФИО и дату до записи.

0:35–1:05. Цели и архитектура
Целями итогового аттестационного задания были генерация паролей по настройкам пользователя, понятная оценка их сложности и сохранение в зашифрованном виде. Дополнительно я предусмотрел поиск записей, отображение даты сохранения и сортировку.
Приложение разделено на модули. Интерфейс отвечает за действия пользователя, отдельные функции выполняют генерацию и оценку, а модуль хранения работает с файлом. Такое разделение позволяет проверять алгоритмы без запуска окна и объяснять каждую часть отдельно.

На экране: Показать схему модулей на слайде 2.

Генерация и оценка

1:05–1:45. Генерация
Для генерации используется стандартный модуль secrets. Пользователь задаёт длину от четырёх до ста двадцати восьми символов и выбирает строчные буквы, прописные буквы, цифры и специальные знаки. Программа строит общий алфавит и выбирает случайные символы. Каждый результат обязательно содержит все выбранные группы. Если кандидату какой-то группы не хватает, программа создаёт новый. В интерфейсе пароль изначально скрыт, его можно показать или скопировать. Нажатие генерации ещё не означает сохранение в файл.

На экране: Сгенерировать учебный пароль длиной 16 и показать настройки.

1:45–2:40. Оценка и применение ИИ
Оценку я реализовал как объяснимую учебную эвристику. Базовая версия учитывала длину и число групп символов. Но этого недостаточно: распространённый шаблон Password123! получал семьдесят шесть баллов и сильную категорию. С помощью Codex я проанализировал такие примеры и добавил распознавание распространённых фрагментов, повторов и последовательностей. Теперь тот же пример получает тридцать девять баллов и слабую категорию с пояснением. На семи заранее заданных основных примерах число совпадений с ожидаемыми категориями выросло с четырёх до семи. Это небольшой учебный набор, а не доказательство безопасности. Приложение не проверяет утечки и не обещает точного времени взлома. Реальные пароли в ИИ не передаются.

На экране: Во вкладке оценки ввести Password123! и нажать «Оценить». Не использовать свой пароль.

Хранилище и работа со списком

2:40–3:40. Зашифрованное хранилище
Для хранения используется библиотека cryptography. При создании файла пользователь задаёт мастер-пароль длиной не менее двенадцати символов и подтверждает его. Из мастер-пароля алгоритм Argon2id получает ключ. Для каждого нового хранилища создаются шестнадцать случайных байтов, которые называются криптографической солью. Она хранится открыто и нужна для повторного получения того же ключа. Сам мастер-пароль и ключ на диск не записываются. Fernet шифрует названия вместе с паролями и проверяет подлинность данных при открытии. Сохранение выполняется через временный файл с шифротекстом. При ошибке записи прежние данные остаются доступными. Работа с файлом выполняется в фоне, чтобы окно продолжало реагировать на действия пользователя.

На экране: Открыть заранее созданное учебное хранилище. При необходимости коротко показать предупреждение о коротком вводе.

3:40–4:15. Поиск даты и закрытие
Когда записей много, их можно найти по части названия без учёта регистра. Поиск не обращается к содержимому паролей. Список сортируется по названию и дате в обе стороны. Отображается момент сохранения записи в часовом поясе текущего компьютера. При фильтрации ранее показанный секрет скрывается. Кнопка «Закрыть хранилище» завершает сеанс и очищает поля. Записи остаются в файле, а для повторного открытия нужен мастер-пароль. Его восстановление и смена в первой версии не предусмотрены.

На экране: Найти учебную запись, сменить сортировку, сбросить поиск и закрыть хранилище.

Проверка и завершение

4:15–4:55. Тестирование и сборка
Для проверки написано сорок семь автоматических тестов. Они охватывают генерацию, оценку, работу ключа и файла, диалог мастер-пароля, поиск и сортировку. Проверяются граничные значения, неверный пароль, повреждение токена и отказ записи. Ручные проверки дополнительно подтвердили работу интерфейса и повторное открытие после перезапуска. Для удобства запуска я подготовил самостоятельный exe для Windows x64 через PyInstaller. В него включены Python, Tkinter и необходимые зависимости. Проверка готового exe из отдельной папки без Python в PATH прошла успешно. Это не заменяет отдельную проверку на другом компьютере.

На экране: Показать сводку 47 тестов и файл exe. Не запускать длинную установку во время записи.

4:55–5:30. Результаты и выводы
Результатами моей работы стали приложение с настраиваемой генерацией, объяснимой оценкой сложности и зашифрованным хранилищем, а также поиск, сортировка и самостоятельная Windows-сборка. В ходе работы я применил функции и классы Python, обработку ошибок, тестирование, Git и инструменты ИИ. Код снабжён учебными комментариями, а ограничения оценки и хранения описаны в документации. Функции выбранного кейса реализованы. Дальнейшее развитие может включать смену мастер-пароля и расширение правил оценки. Спасибо за внимание, доклад окончен.

На экране: Показать итоговый слайд, сохранить лицо в кадре до завершения.